



Institut Supérieur de Comptabilités et D'administration des Entreprises

Nouakchott-Mauritanie



Rapport de stage

Filière : Développement informatique

Stage effectuée du [15/07/2021] au [23/08/2021]

Nom de la société : SYSKAT Technologie



SYSKAT Technologie

Intitulé du Stage :

Développer une application web :

« Gestion des blocs opératoires »

Elaborer Par :

Brahim Mohamed Lemine Rabnay

Encadrer par :

Aboubacar ould Bah

[Année Universitaire 2021-2022]



REMERCIEMENTS

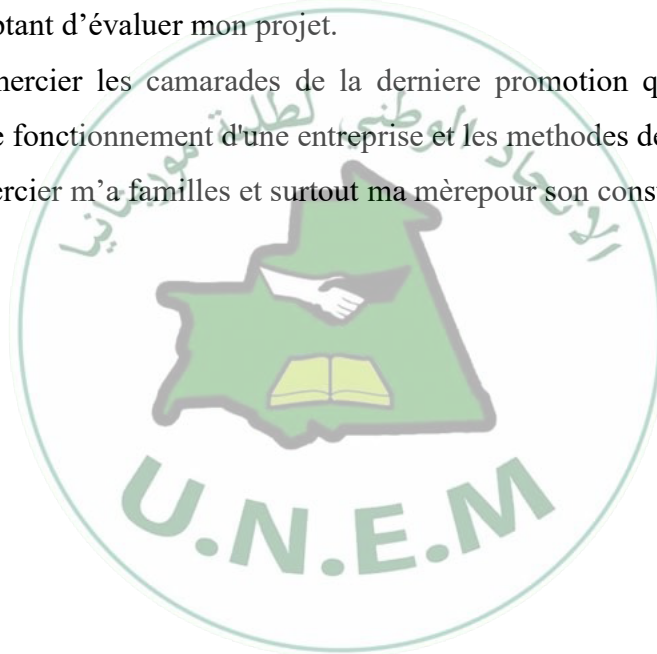
Je n'aurai pas commencé ce rapport sans remercier ALLAH le tout puissant, le tout miséricordieux, qui m'a donné grâce et bénédiction pour faire ce stage.

Mes remerciements s'étendent également à Monsieur **Aboubekrine ould Bah**, Directeur Général de **SYSKAT Technologies**, pour m'avoir donné la chance d'intégrer **SYSKATTechnologie** et qui a su m'aider et me conseiller quand j'en avais besoin. Il est resté très à l'écoute et a su me faire confiance. Aussi mes enseignants durant ces années d'études auprès de qui mon beaucoup appris.

Mes remerciements s'adressent également aux membres de jury pour l'honneur qu'ils m'adressent en acceptant d'évaluer mon projet.

Ainsi je tiens à remercier les camarades de la dernière promotion qui m'ont permis d'en apprendre plus sur le fonctionnement d'une entreprise et les méthodes des recherches

J'aimerais aussi remercier m'a familles et surtout ma mère pour son constant soutien.



LISTE DES FIGURES ET DES TABLEAUX

Figure 1 : Diagrammes UML.....	17
Figure 2 : Diagrammes de cas d'utilisation générale.....	19
Figure 3 : Diagrammes de cas d'utilisation pour gérer les tableaux de bords	19
Figure 4 : Diagramme des cas d'utilisation gestion des opérations.....	20
Figure 5 : Diagrammes de cas d'utilisations gestion des patients.....	20
Figure 6 : Diagrammes de classe	21
Figure 7 : Model logique de données.....	22
Figure 8 : Model MVS	24
Figure 9 : serveur web.....	25
Figure 10 : serveur de base de données.....	26
Figure 11 : logo HTML.....	27
Figure 12 : logo CSS.....	Erreur ! Signet non défini.
Figure 13 : logo JavaScript	28
Figure 14 : logo SQL	29
Figure 15 : logo PHP.....	29
Figure 16 : logo MYSQL.....	30
Figure 17 : logo UML.....	31
Figure 18 : jquery.....	32
Figure 19 : logo Worbench	33
Figure 20 : logo Worbench	33
Figure 21 : logo Worbench	33
Figure 22 : logo Visual Studio	34
Figure 23 : logo Laravel.....	34
Figure 24 : interface d'authentification.....	37
Figure 25 : Interfaces de tableau de bord.....	38
Figure 26 : Interface Chirurgiens.....	38
Figure 27 : Interface de gestion des Anesthésistes.....	39
Figure 28 : Interface patient	39
Figure 29 : interface de gestion des services.....	40
Figure 30 : Interface des Operations	40

TABLE DES MATIÈRES

REMERCIEMENTS	1
LISTE DES FIGURES ET DES TABLEAUX	2
INTRODUCTION :	5
1 L'ENTREPRISE SYSKAT	6
1.1. COMPETENCE DE SYSKAT TECHNOLOGIE	7
1.2. LES SOLUTIONS DE SYSKAT	8
1.3. ORGANIGRAMME DE L'ORGANISATION DE SYSKAT	8
1.4. PRESENTATION DU PROBLEMATIQUE	9
2 ANALYSE ET SPECIFICATION DES BESOINS	10
1. INTRODUCTION :	11
2. SPECIFICATION DES BESOINS :	11
2.1. <i>Besoins fonctionnels :</i>	11
2.2. <i>Besoins non fonctionnels :</i>	11
3. ANALYSE DES BESOINS	12
3.1. <i>Etude préalable :</i>	12
3.2. <i>Identification des acteurs :</i>	12
4. METHODOLOGIE DE DEVELOPPEMENT :	13
4.1.1. <i>Définition de processus unifie :</i>	13
4.1.2. <i>Phase du processus Unifie :</i>	13
4.1.3. <i>Activités du processus unifie</i>	13
5. CONCLUSION :	14
3 CONCEPTION	15
1. PRESENTATION DU LANGAGE UML	16
2. CONCEPTION DE L'APPLICATION	19
2.1. <i>Modélisation avec le diagramme des cas d'utilisation :</i>	19
2.2. <i>DIAGRAMME DE CLASSE</i>	20
2.3. <i>Passage au modèle relationnelle</i>	21
3. CONCLUSION	22
4 DEVELOPEMENT ET REALISATION	23
1. INTRODUCTION :	24
2. ARCHITECTURE DE L'APPLICATION :	24
2.1. <i>Model MVC :</i>	24
3. ARCHITECTURE DU SYSTEME :	25
3.1. <i>Serveur web :</i>	25
3.2. <i>Serveur de bases de données :</i>	26
4. ENVIRONNEMENT DE DEVELOPPEMENT :	26
4.1. <i>Environnement matériels :</i>	27
4.2. <i>Environnement logiciels :</i>	27
4.3. <i>Outils et logiciel</i>	30
4.4. <i>Argumentaire du choix de Laravel</i>	35
5 PRESENTATION	36
DU SYSTEME	36
1. INTRODUCTION :	37



LES PRINCIPALES INTERFACES GRAPHIQUE :	37
2.1. Page d'authentification :	37
2.2. Interfaces de tableaux de bord	38
2.3. Interfaces de "chirurgiennes "	38
2.4. Interfaces de « Gestion des Anesthésistes »	39
2.5. Interfaces de Gestion des Patients	39
2.6. Interface de gestion des services	40
2.7. Interfaces Gestions des opérations	40
3. CONCLUSION :	41
4. CONCLUSION GENERALE :	41
WEBOGRAPHIE	42



INTRODUCTION :

L'informatisation des unités de soins constitue un enjeu affiché pour améliorer l'organisation de la prise en charge des malades. Depuis plus de vingt ans l'informatique a franchi les portes de l'hôpital. Et de nos jours, ce progrès fait l'objet de projets d'informatisation qui ont pour but d'améliorer la gestion de la prise en charge des malades.

Avant l'invention de l'ordinateur, on enregistrerait toutes les informations manuellement sur des supports en papier, ce qui engendrait beaucoup de problèmes tels qu'une perte de temps considérable dans la saisie et la recherche, leur dégradation par une consultation fréquente et leur sécurisation etc.

Ce même problème se projette jusqu'aux établissements hospitaliers, ce qui fait qu'ils font partie intégrante des structures que l'informatique pourra beaucoup aider. C'est sur cette optique que nous nous sommes intéressés aux hôpitaux. Ce choix est motivé par le besoin d'améliorer le système de gestion des patients et les problèmes liés à la gestion des données. Cette dernière a fait que chaque service a son propre système de numérotation, ce qui ne permet pas d'avoir un bon suivi de tous les patients. Par conséquent, on note une redondance récurrente et une perte de temps dans la saisie et la recherche de l'information du patient.

Pour une bonne structuration, j'ai agencé mon travail en cinq chapitres. Au premier chapitre je vais présenter le cadre général de mon stage. Au deuxième et au troisième chapitre je ferai le projet sur l'analyse et la spécification des besoins exprimés afin de passer à la conception. Ensuite, j'ai passé à la réalisation dans le quatrième chapitre pour terminer avec une présentation globale du système en quatrième chapitre.



1 L'ENTREPRISE SYSKAT

SYSKAT Technologies SARL est une société Mauritanienne. Elle serve les particuliers, les petites, moyennes et grandes entreprises, cette entreprise née 2007. Elle est spécialisée dans le domaine de la conception, création et administration des bases de données, Développement des applications, Traitement des données.

Elle propose une offre de services dans le conseil en systèmes d'informations, le développement de logiciels, l'intégration, la conduite de projets et le transfert de compétences. Comme toute Société de services en ingénierie informatique(SSII) qui vise la performance, **SYSKAT** est à l'écoute du marché et cherche toujours à avoir une satisfaction totale du client.

1.1. Compétence de SYSKAT Technologie

❖ Ingénierie



✓ Conception, création et administration des bases de données

✓ Développement des applications

✓ Traitement des données

❖ Web



✓ Stratégie et ergonomie

✓ Rédaction optimisée

✓ Conception

✓ Création

✓ Analyse des résultats

✓ Hébergement Web

❖ Sécurité et administration des réseaux



✓ Gestion de sécurité Firewalls et DMZ

✓ Supervisions de réseaux et de Systèmes d'exploitation

✓ Mise en place de mécanismes d'authentification SSO

✓ Mise en place des réseaux internes Ethernet et WIFI

❖ Technologies



✓ Langages : JAVA, C, C++, SQL, PL/SQL, .NET, PHP.

✓ Platform : Linux, Windows

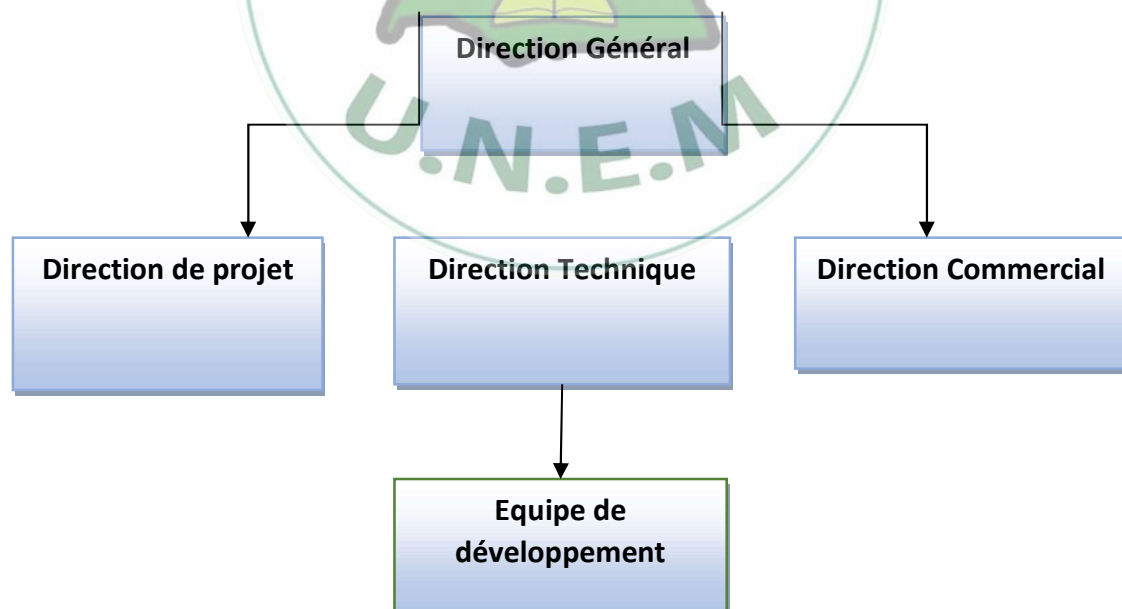
Frameworks: JSF, STRUTS, Hibernate, spring, Design Pattern, Maven, Ant, Zend.

1.2. Les solutions de SYSKAT

SYSKAT a des solutions développées à partir des technologies **JAVA/JEE** avec l'utilisation des Framework modernes tels que **JSF**, Hibernante, **Spring**...

- Système d'Information Hospitalier Intégré « **SIH** »
- Gestion Complète d'une Clinique « **ESSIHA** »
- Gestion des Ressources Humaines « **GRH** »
- Gestion de comptabilité générale « **COMPTA** »
- Gestion commerciale « **GESCOM** »
- Centre messagerie « **AL-MERSAL** »
- etc.

1.3. Organigramme de L'Organisation de SYSKAT



1.4. Présentation du problème

Une bonne gestion de programme opératoire demeure nécessaire et indispensable pour un bon suivi et une bonne prise en charge des malades. Même s'il y'en a dans la plupart des établissements hospitaliers, on a pu constater que ces derniers optent généralement pour les méthodes classiques, c'est-à-dire toutes les informations du patient pour une opération sont enregistrées manuellement. Cette méthode présente beaucoup de limites telles qu'une perte de temps considérable dans la saisie et la recherche de l'information du patient. C'est dans ce cadre que nous avons jugé nécessaire de mettre en place une application pour la gestion des programme opératoires afin d'améliorer cette dernière.



2 ANALYSE ET SPÉCIFICATION DES BESOINS

1. INTRODUCTION :

Le travail qui m'a été confié pendant mon stage consiste à développer une application web qui a pour objectif de gérer le planning d'opérateurs (des patients, des chirurgiens, des Anesthésistes, des opérations ...). Pour se faire il est indispensable de réaliser une étude de ce qui existe déjà sur le marché pour comprendre en premier, comment fonctionnent les applications existantes et, en second, sur quels points je vais travailler. Mais par conséquent il n'y a aucune application web de ce genre sur le marché, Donc la spécification des besoins fonctionnels et non fonctionnels de l'application ont été établis non pas par une étude de l'existant, mais par l'échange d'avis entre moi et mon encadreur.

2. Spécification des besoins :

Dans cette partie, on explique en détail ce que l'application est censée faire et ceci à travers la spécification des besoins fonctionnels et non fonctionnels.

2.1. Besoins fonctionnels :

Les besoins fonctionnels ou besoins métiers représentent les actions que le système doit exécuter.

Cette application doit satisfaire principalement les besoins fonctionnels suivants :

- Gestion de Tableau de bord
- Gestion des Chirurgiens
- Gestion des Services
- Gestion des Opérations
- Gestion des Anesthésistes
- Consultation des Patients
- Gestion des Salles Operations

2.2. Besoins non fonctionnels :

Ce sont des exigences qui ne concernent pas spécifiquement le comportement du système mais plutôt ils identifient des contraintes internes et externes du système. Les principaux besoins non fonctionnels de mon application se résument dans les points suivants :

- **Performance**

- L'application répond à tous les exigences de l'utilisateur d'une manière optimale
- **Fiabilité**
 - Bon fonctionnement de l'application sans détection de défaillance
- **Rapidité**
 - Le déplacement entre les pages sera facile et rapide
- **Sécurité**
 - Les comptes d'utilisateurs sont sécurisés par des mots de passe (longueur, caractères spéciaux, politique de réutilisation...)
 - Déconnexion après un temps d'inactivité
- **Convivialité**
 - Un design clair, souple et interactif
 - Une bonne interface qui donne l'envie à l'utilisateur d'utiliser l'application
 - Positionnement du contenu dans les pages de la manière la plus accessible
- **Portabilité**
 - L'application est multiplateforme : Elle fonctionne sur tout système d'exploitation
 - Elle fonctionne sur tout type de terminal

3. Analyse des besoins

L'objectif de l'analyse est d'accéder à une compréhension des besoins et des exigences du client. Il s'agit de livrer des spécifications pour permettre de choisir la conception de la solution.

3.1. Etude préalable :

L'étude préalable est une partie primordiale dans la réalisation d'un projet informatique. Pour atteindre nos buts et objectifs, il est nécessaire d'avoir une vue claire des différents besoins escompter. Raison pour laquelle on est appelé à faire une étude des méthodes ou systèmes qu'utilise l'hôpital pour la gestion des programmes opératoires.

3.2. Identification des acteurs :

Un acteur représente l'abstraction d'un rôle joué par des entités externe (utilisateur, dispositif matériel ou autre système) qui interagissent directement avec le système étudié. Dans mon application le Chirurgienne et Assistante sont les seuls acteurs qui interagissent avec le système.

4. Méthodologie de développement :

Contenu de la nature de mon projet, j'ai jugé que la méthode PU (processus unifié) serait la plus adapté pour sa réalisation.

4.1.1. Définition de processus unifié :

Le processus unifié est un processus de développement logiciel itératif, centré sur l'architecture, piloté par des cas d'utilisation et orienté vers la diminution des risques.

4.1.2. Phase du processus Unifié :

La méthode UP se base sur quatre phases :

- **Analyse des besoins** : Établir une vision globale du projet où on spécifie Les besoins et on étudie la faisabilité du projet.
- **Élaboration** : On reprend les éléments de l'analyse des besoins et on développe une architecture de référence, les risques et la plupart des besoins sont identifiés.
- **Construction** : Finaliser l'analyse, la conception, l'implémentation et les tests puis transformer l'architecture de référence en produit exécutable tout en veillant à respecter son intégrité.
- **Transition** : Livraison du produit au client afin d'effectuer des essais pour détecter d'éventuelles anomalies.

4.1.3. Activités du processus unifié

Chaque phase est constituée d'une succession d'activités. Les activités de la méthode UP sont :

- **Expression des besoins** : Compréhension et expression des besoins et des exigences du client qu'elles soient fonctionnelles ou non fonctionnelles.
- **Analyse et Conception** : Permet d'acquérir une compréhension approfondie des contraintes liées aux outils de réalisation en prenant en compte le choix d'architecture technique retenu pour le développement et l'exploitation système.
- **Implémentation** : On implémente le système sous forme de composants, bibliothèques et de fichiers. Elle a pour objectif de planifier l'intégration.

Tests : Permettent de vérifier les résultats de l'implémentation de toutes les exigences et de s'assurer de la bonne intégration de tous les composants dans le logiciel.

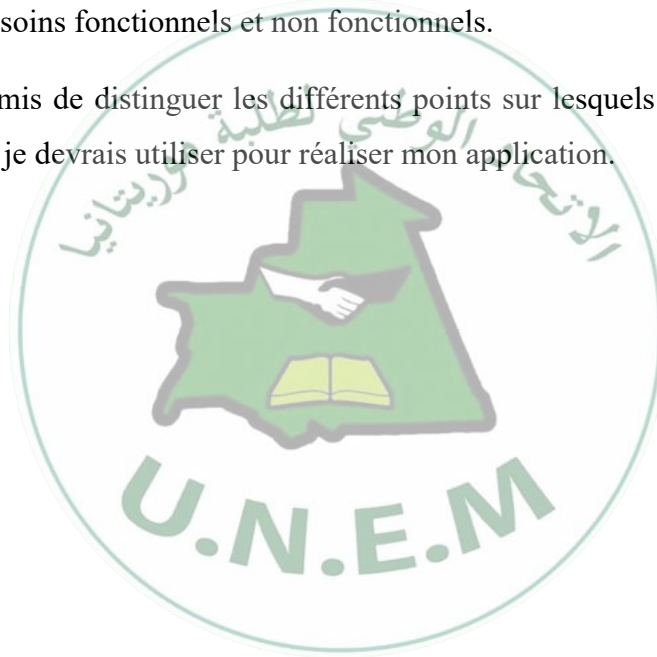
➤ **Déploiement :** Livraison et exploitation du produit

5. Conclusion :

Ce chapitre a été dédié à la spécification des besoins après une étude détaillée de l'application : son objectif et son fonctionnement.

J'ai consacré la deuxième semaine de mon stage à cette étude ce qui m'a donné une bonne conscience de ses besoins fonctionnels et non fonctionnels.

Cette étude m'a permis de distinguer les différents points sur lesquels je vais travailler et les technologies que je devrais utiliser pour réaliser mon application.



3 CONCEPTION



La démarche de conception est une étape fondamentale dans le processus de développement puisqu'elle fait correspondre la vision applicative (le modèle d'analyse) à la vision technique (l'environnement de développement et d'exécution).

Ce chapitre vise à illustrer la phase de conception et les modèles **UML** associés. J'ai commencé par une présentation du langage **UML**, et par la suite établir les diagrammes de séquences, des cas d'utilisation, ensuite et le diagramme de classe, suivi du passage de ce dernier en Modèle Logique de Données (**MLD**) et je termine avec une petite conclusion.

1. Présentation du langage UML



Pour faire face à la complexité des systèmes d'information, de nouvelles méthodes et outils ont été créés. La principale avancée des quinze dernières années réside dans la programmation orientée objet.

Face à ce nouveau mode de programmation, les méthodes de modélisation classique (telle que **MERISE**) ont rapidement montré certaines limites et ont dû s'adapter.

De très nombreuses méthodes de modélisation ont également vu le jour comme **Boch**, **OMT** ...

Dans ce contexte et devant le foisonnement de nouvelles méthodes de conception orientée objet, l'**OMG** (Object Management Group) a eu comme objectif de définir une notation standard utilisables dans les développements informatiques basés sur l'objet. C'est ainsi qu'est apparu **UML** (qui signifie en français langage de modélisation unifiée), qui est issu de la fusion des méthodes de **Booch**, **OMT** et **OOSE**. C'est un langage de modélisation qui permet de représenter graphiquement les besoins des utilisateurs à l'aide de diagramme.

Un diagramme **UML** est une représentation graphique, qui s'intéresse à un aspect précis du modèle. C'est une perspective du modèle, pas « le modèle ». Chaque type de diagramme **UML** possède une structure.

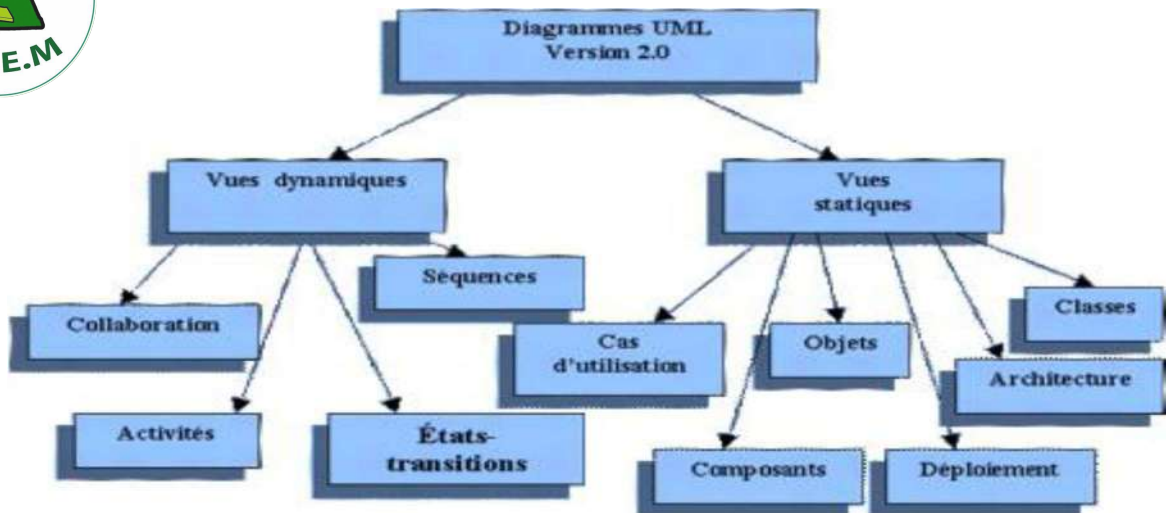


Figure 1 : Diagrammes UML

Il existe deux types de vues du système constituant chacune des diagrammes qui sont répartis selon leurs aspects statiques ou dynamiques.

Selon les vues statiques nous avons :

- Le diagramme des cas d'utilisation

Le diagramme de cas d'utilisation permet de structurer les besoins des utilisateurs et les objectifs correspondants d'un système. Il essaiera de répondre à des questions du genre : Qui devra pouvoir faire quoi grâce au logiciel. Les acteurs principaux qui sont liés à un package auront besoin de cette partie du logiciel pour réaliser une plusieurs lots d'actions.

- Le diagramme d'objet

Le diagramme d'objet sert à illustrer les classes complexes en utilisant des exemples d'instances. A l'exception de la multiplicité, qui est explicitement indiquée, le diagramme d'objets utilise les mêmes concepts que le diagramme de classes. Ils sont essentiellement utilisés pour comprendre ou illustrer des parties complexes d'un diagramme de classes.

- Le diagramme des classes

Le diagramme des classes représente les entités manipulées par les utilisateurs. L'intérêt du diagramme des classes est de modéliser les entités du système d'information. Le diagramme met en évidence d'éventuelles relations entre les classes ou les entités.

- Le diagramme de déploiement

Les diagrammes de déploiement correspond à la description de l'environnement d'exécution du système (matériel, réseau...) et de la façon dont les composants y sont installés. Les diagrammes de déploiement sont donc très utiles pour modéliser l'architecture physique d'un système.

les vues dynamiques nous avons :

- Le diagramme de collaboration

Le diagramme de collaboration (appelé également diagramme de communication) permet de mettre en évidence les échanges de messages entre objets. Cela nous aide à voir clair dans les actions qui sont nécessaires pour produire ces échanges de messages. Et donc de compléter, si besoin, les diagrammes de séquence et de classes.

- Le diagramme de séquence

Le diagramme de séquence permet de décrire les différents scénarios d'utilisation du système. C'est une variante du diagramme de collaboration sauf qu'il possède intrinsèquement une dimension temporelle mais ne représente pas explicitement les liens entre les objets.

- Le diagramme d'états-transitions

Le diagramme d'état-transition permet de décrire le cycle de vie des objets d'une classe. Il définit l'enchaînement des états de classe et font donc apparaître l'ordonnancement des travaux.

- Le diagramme d'activités

Le diagramme d'activité représente le déroulement des actions, sans utiliser les objets. En phase d'analyse, il est utilisé pour consolider les spécifications d'un cas d'utilisation.

Conscient de la complémentarité qui existe entre ces différents diagrammes, et sachant qu'UML ne préconise aucune démarche, on a jugé nécessaire de ne prendre que certains de ces diagrammes qui répondent bien à nos besoins afin de concevoir notre système.

Cependant, les diagrammes qui vont être utilisés dans la suite de mon exposé sont les suivants :

Le diagramme des cas d'utilisation ;

Et Le diagramme des classes.

2. CONCEPTION DE L'APPLICATION

2.1. Modélisation avec le diagramme des cas d'utilisation :

a. Diagramme des cas d'utilisation général

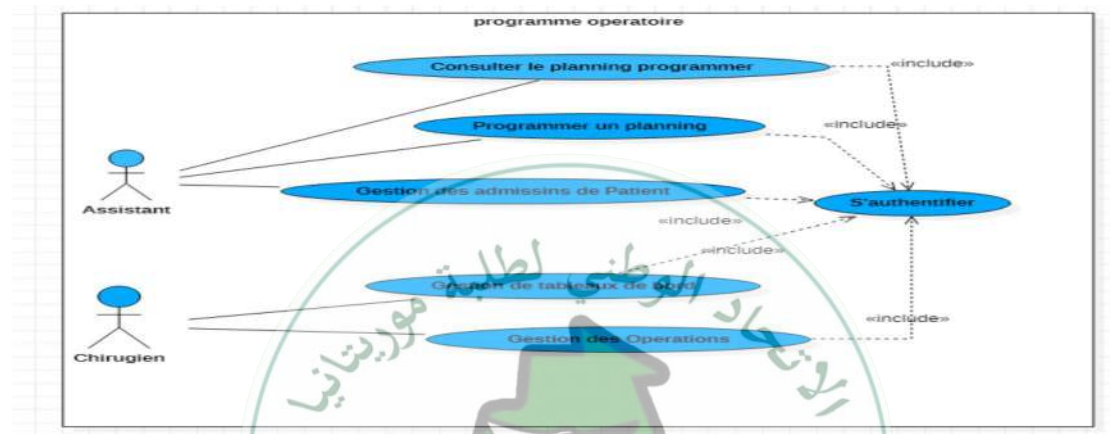


Figure 2 : Diagrammes de cas d'utilisation générale

b. Diagramme des cas d'utilisation gestion de tableau de bord

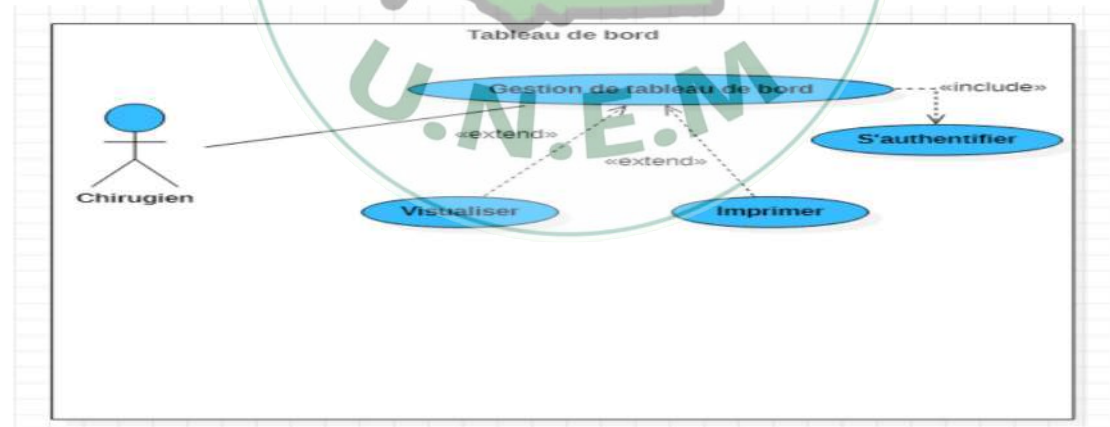


Figure 3 : Digrammes de cas d'utilisation pour gérer les tableaux de bords

c. Diagramme des cas d'utilisation gestion des Opération

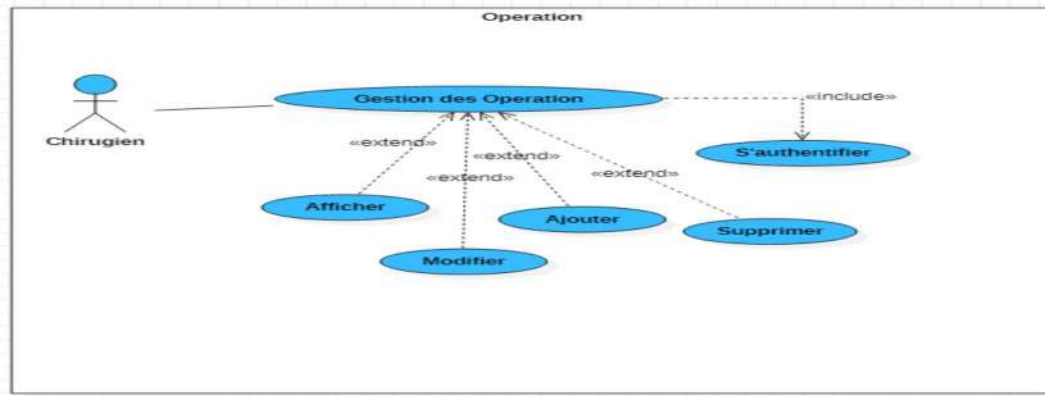


Figure 4 : Diagramme des cas d'utilisation gestion des opérations

d. Diagramme des cas d'utilisation gestion des Patients



Figure 5 : Diagrammes de cas d'utilisations gestion des patients

2.2. DIAGRAMME DE CLASSE

Le diagramme de classes exprime la structure statique du système en termes de classes et de relations entre eux; Son intérêt est de modéliser les entités du système d'information.

Le diagramme de classe permet de représenter l'ensemble des informations finalisées qui sont gérées par le domaine. Ces informations sont structurées, c'est-à-dire quelles sont regroupées dans des classes.

Le diagramme met en évidence d'éventuelles relations entre ces classes. Le diagramme de classes de mon application est le suivant

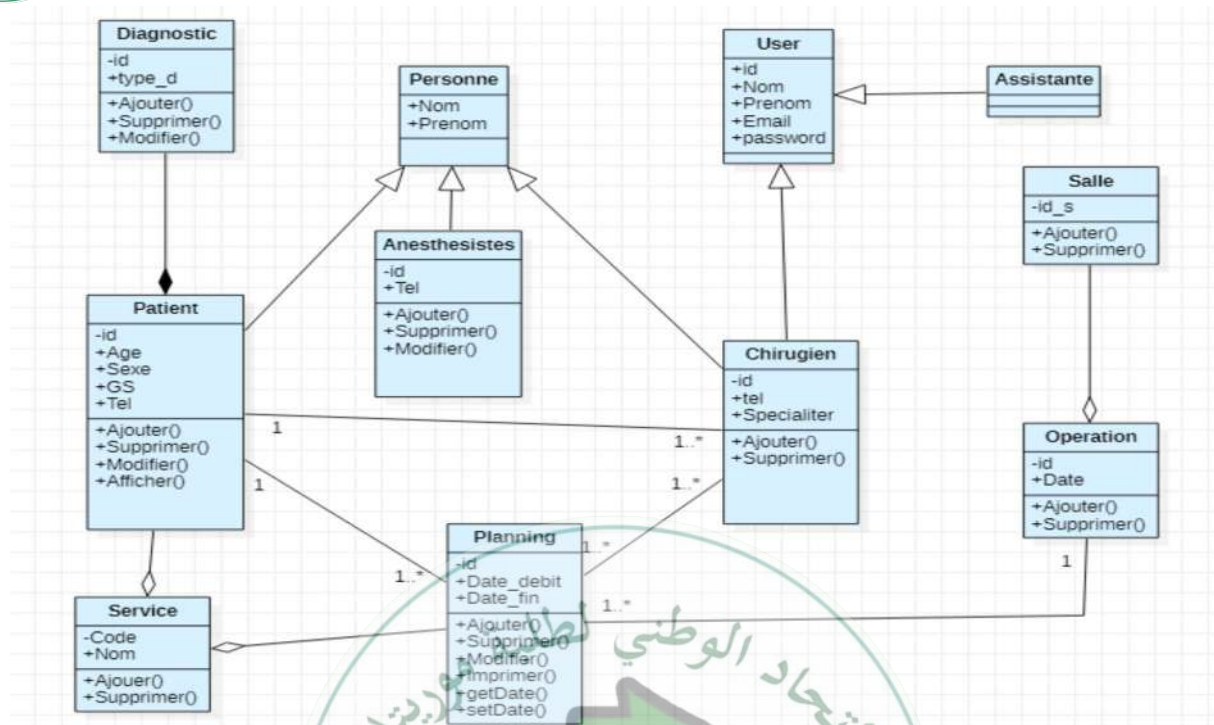


Figure 6 : Diagrammes de classe

2.3. Passage au modèle relationnelle

Le modèle relationnel est le modèle logique de donnée qui correspond à l'organisation des données dans les bases de données relationnelles.

Un modèle relationnel est composé de relations, encore appelée table. Ces tables sont décrites par des attributs aux champs. Pour décrire une relation, on indique tout simplement son nom, suivi du nom de ses attributs entre parenthèses.

L'identifiant d'une relation est composé d'un ou plusieurs attributs qui forment la clé primaire. Une relation peut faire référence à une autre en utilisant une clé étrangère, qui correspond à la clé primaire de la relation référencée.

Le modèle logique de mon application est le suivant :

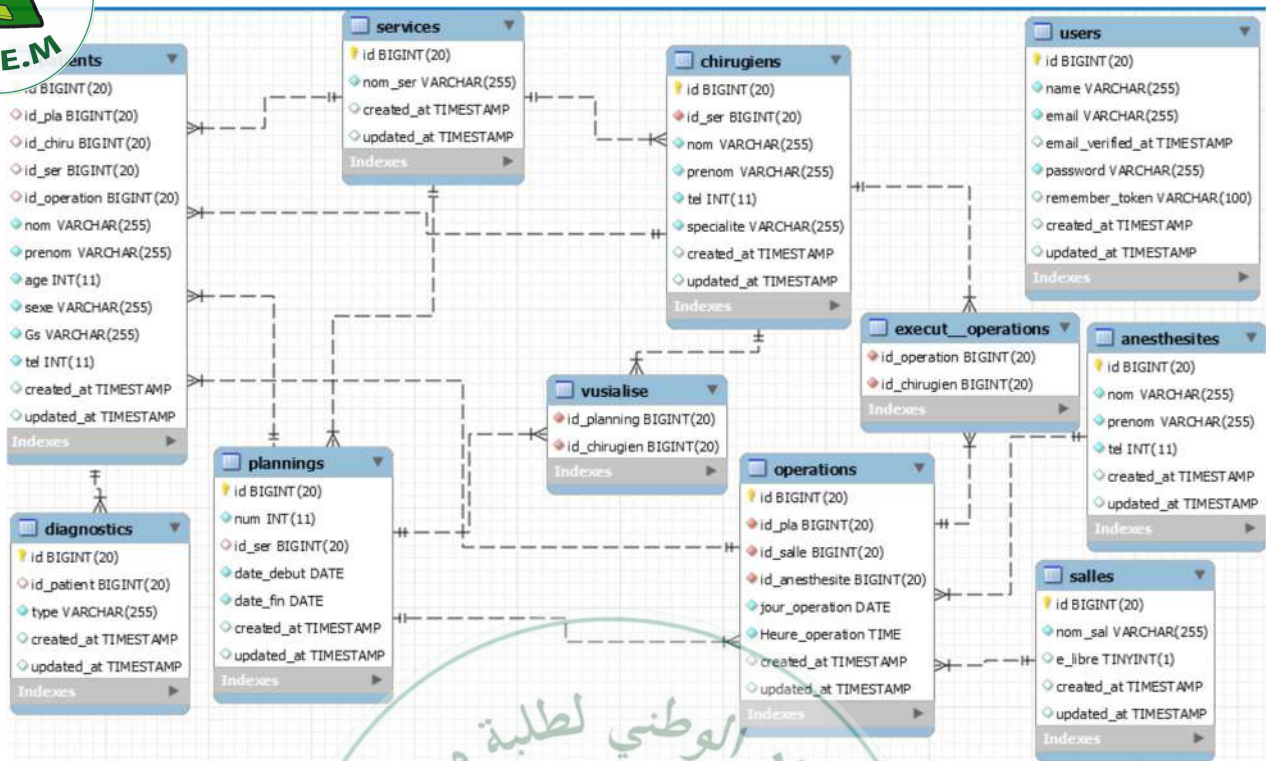


Figure 7: Model logique de données

3. Conclusion

Dans ce chapitre, j'ai présenté mon étude conceptuelle du système. La vue dynamique nous a permis d'avoir une vue générale sur le déroulement des cas d'utilisation et leurs exécutions, cette vue a été modélisée par des diagrammes de séquence du système puis par des diagrammes d'activités.

La vue statique, réalisée par le diagramme des classes nous a permis de définir la structure du système et de dégager les différentes entités y composés.

Enfin la conception graphique nous a permis de représenter les différentes sortes de l'application.



4 DEVELOPEMENT ET REALISATION

1. INTRODUCTION :

Après l'étape de conception de l'application, nous allons dans ce chapitre, d'écrire la phase de réalisation et de développement. Nous allons présenter, en premiers lieu, L'architecture de l'application et du Système ensuite parlerons de l'environnement du travail utilisé pour le développement de l'application.

2. Architecture de l'application :

2.1. Model MVC :

Le développement de mon application s'est basé sur l'architecture **MVC**. Ce paradigme divise l'**THM** en un modèle, une vue et un contrôleur, chacun ayant un rôle bien précis.

Cette architecture permet de bien organiser son code source en nous aidant à savoir quels fichiers créer, mais surtout à définir leur rôle.

Ce maître d'architecture impose la séparation entre les données, la présentation et les traitements, ce qui nous donne trois parties fondamentales dans l'application : le modèle, la vue et le contrôleur.

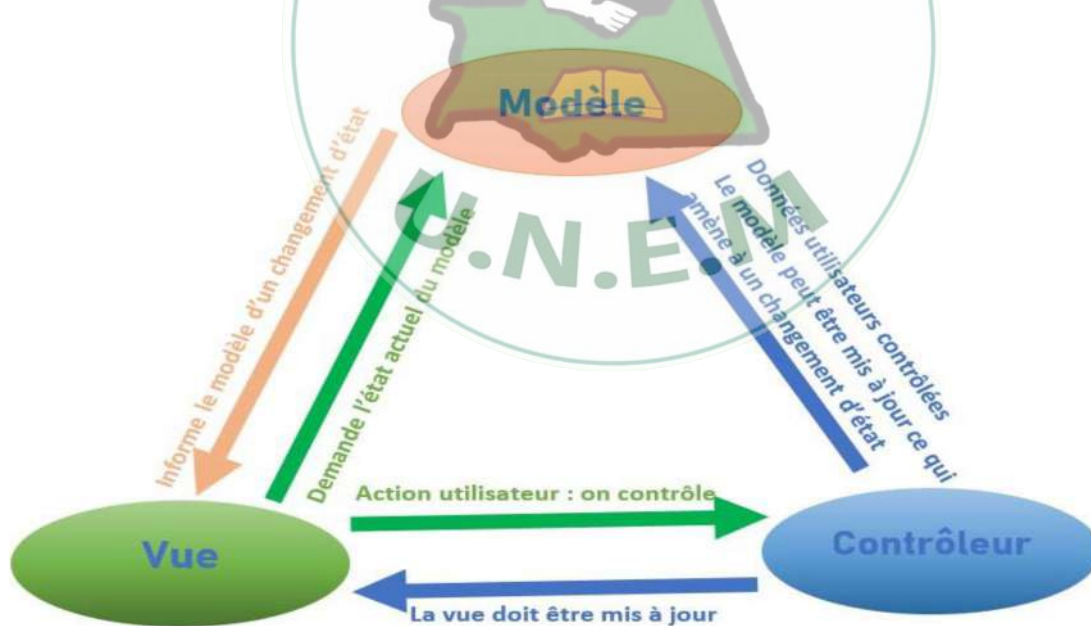


Figure 8 : Model MVS

2.1.1. Modèle :

Cette partie illustre le comportement de notre application : traitement des données et interaction avec la base de données. Son rôle est d'aller récupérer les informations « brutes » dans la base de données, de les organiser et de les assembler pour qu'elles puissent ensuite

traitées par le contrôleur. Le modèle offre des méthodes pour mettre à jour les données (ajout, suppression, modification). On y trouve donc entre autres les requêtes **SQL**.

2.1.2. Vue :

Cette partie se concentre sur l'affichage et correspond à l'interface avec laquelle l'utilisateur interagit. Elle ne fait presque aucun calcul et se contente de récupérer des variables pour savoir ce qu'elle doit afficher. On y trouve essentiellement du code **HTML** mais aussi quelques boucles et conditions **PHP** très simples, pour afficher par exemple une liste de messages.

2.1.3. Contrôleur :

Cette partie gère la logique du code qui prend des décisions. C'est en quelque sorte l'intermédiaire entre le modèle et la vue : le contrôleur va demander au modèle les données, les analyser, prendre des décisions et renvoyer le texte à afficher à la vue. Le contrôleur contient exclusivement du **PHP**. C'est notamment lui qui détermine si le visiteur a le droit de voir la page ou non (gestion des droits d'accès).

3. Architecture du système :

L'architecture client-serveur s'appuie sur un poste central, le serveur, qui envoie des données aux machines clientes.

3.1. Serveur web :

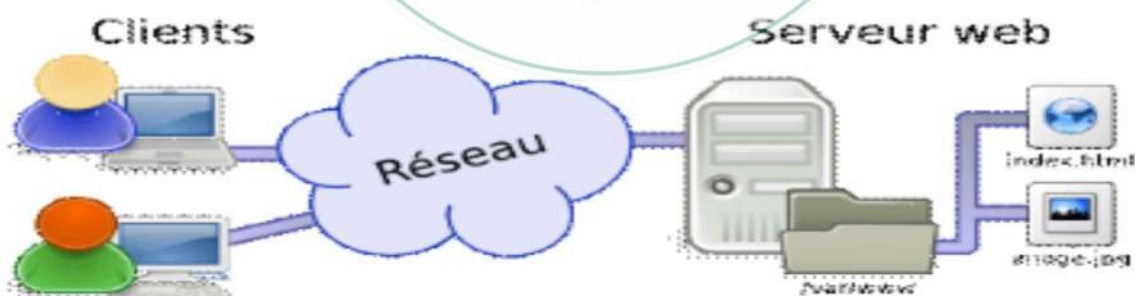


Figure 9 : serveur web

Un serveur web peut faire référence à des composants logiciels (**software**) ou à des composants matériels (**hardware**) ou à des composants logiciels et matériels qui fonctionnent ensemble.

Au niveau des composants matériels, un serveur web est un ordinateur qui stocke les fichiers qui composent un site web (par exemple les documents **HTML**, les images, les feuilles de

CSS, les fichiers **JavaScript**) et qui les envoie à l'appareil de l'utilisateur qui visite le site. Cet ordinateur est connecté à Internet et est généralement accessible via un nom de domaine tel que **mozilla.org**.

Au niveau des composants logiciels, un serveur web contient différents fragments qui contrôlent la façon dont les utilisateurs peuvent accéder aux fichiers hébergés. On trouvera à minima un serveur **HTTP**. Un serveur **HTTP** est un logiciel qui comprend les **URL** et le protocole **HTTP**.

3.2. Serveur de bases de données :



Figure 10 : serveur de base de données

Un serveur de base de données sert à stocker, à extraire et à gérer les données dans une base de données. Il permet également de gérer la mise à jour des données. Il donne un accès simultané à cette base à plusieurs serveurs web et utilisateurs. Enfin, il assure la sécurité et l'intégrité des données. Et quand on parle de données, on entend peut-être des millions d'éléments simultanément accessibles à des milliers d'utilisateurs.

En plus de ces fonctions principales, le logiciel de serveur de base de données offre des outils qui facilitent et accélèrent l'administration de la base, comme l'exportation de données, la configuration de l'accès de l'utilisateur et la sauvegarde des données.

4. Environnement de Développement :

Pour interagir avec le serveur et la base de données, nous sommes appelés à faire recours au moins à un langage de programmation. Dans cette partie j'ai identifié les différentes caractéristiques de l'environnement matériel et logiciel que j'aiservi à l'implémentation de mon application.

1. Environnement matériels :

La machine utilisée pour réaliser ce projet :

Ordinateur portable



Machine DELL

Mémoire Vive : 8 Go

Disque Dur : 300 Go

Processeur : Intel (R)

Type de système : Windows 10

4.2. Environnement logiciels :

4.2.1. Les langages de programmation

- **HTML**



Figure 11 : logo HTML

L'HyperText Mark up Langage, généralement abrégé **HTML**, est le langage de balisage conçu pour représenter les pages web. C'est un langage permettant d'écrire de l'hypertexte, d'où son nom. **HTML** permet également de structurer sémantiquement et logiquement et de mettre en forme le contenu des pages, d'inclure des ressources multimédias dont des images, des formulaires de saisie et des programmes informatiques.

permet de créer des documents interopérables avec des équipements très variés de manière conforme aux exigences de l'accessibilité du web. Il est souvent utilisé conjointement avec le langage de programmation JavaScript et des feuilles de style en cascade (CSS). **HTML** est inspiré du Standard GeneralizedMark upLanguage (SGML). Il s'agit d'un format ouvert.

- **CSS**



Figure 12 : logo CSS

Les feuilles de style en cascade¹, généralement appelées **CSS** de l'anglais Cascading Style Sheets, forment un langage informatique qui décrit la présentation des documents **HTML** et **XML**. Les standards définissant **CSS** sont publiés par le World Wide Web Consortium (W3C). Introduit au milieu des années 1990, **CSS** devient couramment utilisé dans la conception de sites web et bien pris en charge par les navigateurs web dans les années 2000.

- **Java Script**



Figure 13 : logo JavaScript

JavaScript est un langage de programmation de scripts principalement employé dans les pages web interactives mais aussi pour les serveurs avec l'utilisation (par exemple) de **Node.js**. C'est un langage orienté objet à prototype, c'est-à-dire que les bases du langage et ses principales interfaces sont fournies par des objets qui ne sont pas des instances de classes, mais qui sont chacun équipés de constructeurs permettant de créer leurs propriétés, et notamment une propriété de prototypage qui permet d'en créer des objets héritiers personnalisés. En outre, les fonctions sont des objets de première classe. Le langage supporte le paradigme objet, impératif et fonctionnel. **JavaScript** est le langage possédant le plus large écosystème grâce à son gestionnaire de dépendances **NPM**, avec environ 500 000 paquets en août 2017.

JavaScript a été créé en 1995 par Brendan Eich. Il a été standardisé sous le nom **ECMAScript** en juin 1997 par **Ecma** International dans le standard **ECMA-262**. Le standard **ECMA-262** en est actuellement à sa 8e édition. JavaScript n'est depuis qu'une implémentation **d'ECMAScript**, celle mise en œuvre par la fondation Mozilla. L'implémentation **d'ECMAScript** par Microsoft (dans Internet Explorer jusqu'à sa version 9) se nomme **JScript**, tandis que celle d'Adobe Systèmes se nomme Action Script.

Avec les technologies **HTML** et **CSS**, **JavaScript** est parfois considéré comme l'une des technologies cœur du World Wide Web. Le langage **JavaScript** permet des pages web interactives, et à ce titre est une partie essentielle des applications web. Une grande majorité des sites web l'utilisent, et la majorité des disposent d'un moteur **JavaScript** dédié pour l'interpréter, indépendamment des considérations de sécurité qui peuvent se poser dans le cas échéant.

- **SQL**



Figure 14 : logo SQL

SQL est un langage informatique utilisé pour effectuer des requêtes sur des bases de données ou systèmes d'information. Il permet d'obtenir les données vérifiant certaines conditions (on parle de critères de sélection). Les données peuvent être triées, elles peuvent également être regroupées suivant les valeurs d'une donnée particulière.

SQL est Le langage de requête le plus connu et le plus utilisé.

- **PHP**



Figure 15 : logo PHP

PHP est un langage informatique utilisé sur l'internet. Le terme PHP est un acronyme récursif de « **PHP : HyperTextPreprocessor** ».

ce langage est principalement utilisé pour produire un site web dynamique. Il est courant que ce langage soit associé à une base de données, tel que **MySQL**. Exécuté du côté serveur (l'endroit où est hébergé le site) il n'y a pas besoin aux visiteurs d'avoir des logiciels ou plugins particulier. Néanmoins, les webmasters qui souhaitent développer un site en **PHP** doivent s'assurer que l'hébergeur prend en compte ce langage.

Lorsqu'une page **PHP** est exécutée par le serveur, alors celui-ci renvoie généralement au client (aux visiteurs du site) une page web qui peut contenir du **HTML**, **XHTML**, **CSS**, **JavaScript**...

4.2.2. *Argumentaire du choix de PHP*

Tout d'abord, **PHP** est gratuit et ne nécessite aucune licence d'utilisation. Ensuite, **PHP** est le langage de programmation Web le plus utilisé au monde. Il existe une communauté de développeurs très active qui rend disponibles des dizaines de milliers de librairies **PHP** de grande qualité ainsi qu'une vaste quantité de documentation et tutoriels accessible à tous au bénéfice de chacun. Ces ressources facilitent notre travail et réduisent notre temps de programmation ce qui se traduit aussi en économie pour le client.

En termes de rapidité et d'efficacité, **PHP** n'a rien à envier aux autres langages. Plusieurs portails très populaires et nécessitant beaucoup de performance l'utilisent.

Nous avons qu'à penser à Facebook, Wikipédia, le réseau CBC, l'université Harvard, pour ne nommer que ceux-là. D'ailleurs le populaire système de gestion de contenu **WordPress** est lui-même construit en **PHP**. Ce dernier représente à lui seul environ 80% des sites Internet sur la toile. Une si grande popularité n'est certainement pas une coïncidence !

4.3. Outils et logiciel

- **MySQL**



Figure 16 : logo MYSQL

MySQL est un Système de Gestion de Base de Données (**SGBD**) parmi les plus populaires au monde. Il est distribué sous double licence, une licence publique générale **GNU** et une propriétaire selon l'utilisation qui en est faite. La première version de MySQL est apparue en 1995 et l'outil est régulièrement entretenu.

Ce système est particulièrement connu des développeurs pour faire partie des célèbres quatuors: **WAMP** (Windows, Apache, **MySQL** et **PHP**), **LAMP** (Linux) et **MAMP** (Mac).

Ces packages sont si populaires et simples à mettre en œuvre que **MySQL** est largement connu et exploité comme système de gestion de base de données pour des applications utilisant **PHP**. C'est d'ailleurs pour cette raison que la plupart des hébergeurs web proposent **PHP** et **MySQL**.

Caractéristiques

MySQL est un serveur de base de données relationnelles **SQL** qui fonctionne sur de nombreux systèmes d'exploitation (dont Linux, Mac OS X, Windows, Solaris, FreeBSD...) et qui est accessible en écriture par de nombreux langages de programmation, incluant notamment **PHP**, **Java**, **Ruby**, **C**, **C++**, **.NET**, **Python** ...

L'une des spécificités de **MySQL** c'est qu'il inclut plusieurs moteurs de bases de données et qu'il est par ailleurs possible au sein d'une même base de définir un moteur différent pour les tables qui composent la base. Cette technique est astucieuse et permet de mieux optimiser les performances d'une application. Les 2 moteurs les plus connus étant **MyISAM** (moteur par défaut) et **InnoDB**.

La réplication est possible avec **MySQL** et permet ainsi de répartir la charge sur plusieurs machines, d'optimiser les performances ou d'effectuer facilement des sauvegardes des données.

- **UML**



Figure 17 : logo UML

UML ("Unified Modeling Language," ou "langage de modélisation objet unifié") est né de la fusion des trois méthodes qui ont le plus influencé la modélisation objet au milieu des années 90 : **OMT**, **Booch** et **OOSE**.

Issu "du terrain" et fruit d'un travail d'experts reconnus, **UML** est le résultat d'un large consensus. De très nombreux acteurs industriels de renom ont adopté **UML** et participent à son développement.

space d'une poignée d'années seulement, **UML** est devenu un standard incontournable. La presse spécialisée foisonne d'articles exaltés et à en croire certains, utiliser les technologies objet sans **UML** relève de l'hérésie. Lorsqu'on possède un esprit un tant soit peu critique, on est en droit de s'interroger sur les raisons qui expliquent un engouement si soudain et massif ! **UML** est-il révolutionnaire ? L'approche objet est pourtant loin d'être une idée récente.

Simula, premier langage de programmation à implémenter le concept de type abstrait à l'aide de classes, date de 1967 ! En 1976 déjà, **Smalltalk** implémente les concepts fondateurs de l'approche objet : encapsulation, agrégation, héritage. Les premiers compilateurs **C++** datent du début des années 80 et de nombreux langages orientés objets "académiques" ont étayés les concepts objets (Eiffel, Objective C, **Loops...**).

Il y a donc déjà longtemps que l'approche objet est devenue une réalité. Les concepts de base de l'approche objet sont stables et largement éprouvés. De nos jours, programmer "objet", c'est bénéficier d'une panoplie d'outils et de langages performants. L'approche objet est une solution technologique incontournable.

• jQuery



Figure 18 : jquery

jQuery est une bibliothèque **JavaScript** libre qui porte sur l'interaction entre **JavaScript** (comprenant Ajax) et **HTML**, et a pour but de simplifier des commandes communes de **JavaScript**. La première version date de janvier 2006.

La bibliothèque contient notamment les fonctionnalités suivantes :

- Parcours et modification du **DOM** (y compris le support des sélecteurs **CSS** 1 à 3 et un support basique de **XPath**) ;
- Événements ;
- Effets visuels et animations ;
- Manipulations des feuilles de style en cascade (ajout/suppression des classes, d'attributs...) ;
- Ajax

Plugins

- Utilitaires (version du navigateur web...)

- **MySQL Workbench**



Figure 19 : logo Worbench

MySQL Workbench est un outil visuel unifié destiné aux architectes de bases de données, aux développeurs et aux administrateurs de base de données. MySQL Workbench fournit la modélisation de données, le développement **SQL** et des outils d'administration complets pour la configuration du serveur, l'administration des utilisateurs, la sauvegarde, etc. **MySQL Workbench** est disponible sous **Windows**, **Linux** et **Mac OS X**.

- **StarUML**



Figure 20 : logo Worbench

StarUML est un logiciel de modélisation UML, qui a été cédé comme open source par son éditeur, à la fin de son exploitation commerciale (qui visiblement continue ...), sous une licence modifiée de **GNU GPL**.

Aujourd'hui la version **StarUML V3** n'existe qu'en licence propriétaire. **StarUML** gère la plupart des diagrammes spécifiés dans la norme **UML 2.0**.

StarUML est écrit en Delphi, et dépend de composants Delphi propriétaires (non open-source).

- **Bootstrap**



Figure 21 : logo Worbench

Bootstrap est un Framework gratuit et à code source ouvert destiné au développement Web frontal réactif et premier mobile. Il contient des modèles de conception basés sur CSS et (éventuellement) JavaScript pour la typographie, les formulaires, les boutons, la navigation et d'autres composants d'interface.

Bootstrap est le troisième projet le plus étoilé sur **Git Hub**, avec plus de 131000 étoiles, derrière seulement **freeCodeCamp** (près de 300 000 étoiles) et légèrement derrière le cadre de **Vue.js**.

- **Visual Studio Code**



Figure 22 : logo Visual Studio

Visual Studio Code est un éditeur de code multiplateforme édité par Microsoft. Cet outil destiné aux développeurs supporte plusieurs dizaines de langages de programmation comme le **HTML**, **C++**, **PHP**, **JavaScript**, **Markdown**, **CSS**, etc.

Visual Studio Code intègre plusieurs outils facilitant la saisie de code par les développeurs comme la coloration syntaxique ou encore le système d'auto-complétion

IntelliSense. En outre, l'outil permet aux développeurs de corriger leur code et de gérer

Les différentes versions de leurs fichiers de travail puisqu'un module de débogage est aussi de la partie.

- **Laravel**



Figure 23 : logo Laravel

Créé en 2011 par **Taylor Otwell**, Laravel est devenu le Framework **PHP** libre et open source le plus populaire au monde. Il est utilisé dans le développement d'application web tout en suivant l'architecture **MVC** et basé sur **Symfony**. Comme chaque version de laravel est documentée de manière détaillée, on peut l'apprendre facilement et compter sur une mise à jour continue de sa base de connaissances.

Laravel est conçu pour le développement rapide d'applications. L'infrastructure dispose d'un moteur de Template bien construit qui permet une grande variété de tâches courantes telles

l'authentification, la mise en cache, la gestion des utilisateurs et le routage **RESTful** pour faciliter la tâche des développeurs de logiciels.

4.4. Argumentaire du choix de Laravel

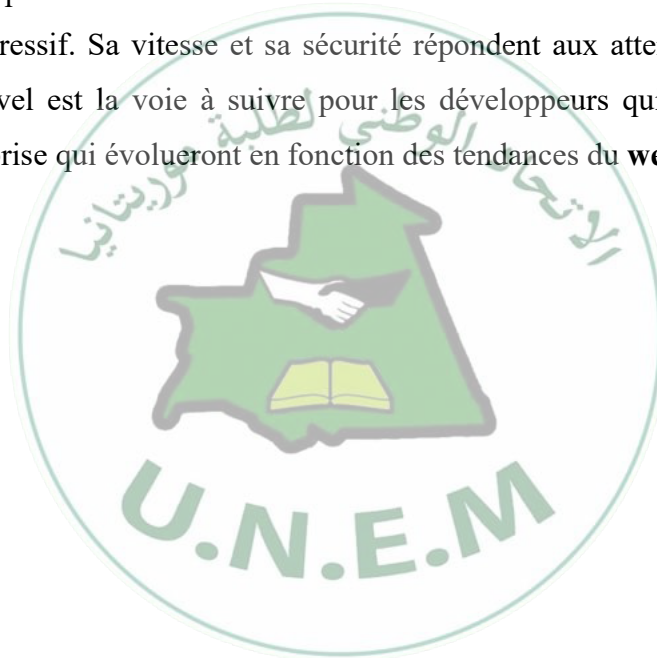
La syntaxe raffinée, la possibilité d'exécuter des tâches en arrière-plan de manière asynchrone pour améliorer les performances et l'intégration bien établie avec Amazon Web (**AWS**) s'ajoute à la liste des fonctionnalités qui font de Laravel l'un des meilleurs frameworks PHP.

Pour la réalisation de mon projet je l'ai choisi pour plusieurs raisons :

Laravel convient au développement d'application avec des besoins complexes, qu'ils soient petits ou grands.

C'est un Framework **PHP** complet avec des fonctionnalités qui nous aideront à personnaliser des applications complexes.

Laravel est très expressif. Sa vitesse et sa sécurité répondent aux attentes d'une application web moderne. Laravel est la voie à suivre pour les développeurs qui souhaitent créer des applications d'entreprise qui évolueront en fonction des tendances du **web**.





1. INTRODUCTION :

Au cours de ce chapitre j'essaierai de faire une présentation générale de l'application. Cette présentation s'articulera sur les principaux types d'interfaces utilisateurs proposés par mon système. Rappelons qu'une interface utilisateur est une partie spécifique de l'application destinée à un utilisateur. Cette partie lui permettra d'interagir avec le système et de profiter aux différentes fonctionnalités qui lui sont offertes.

2. Les Principales Interfaces graphique :

2.1. Page d'authentification :

La page d'authentification peut être considérée comme la page d'accueil principal. Cette partie de l'application permet aux différents utilisateurs de pouvoir s'identifier grâce à un email et un mot de passe afin d'accéder à leur interface graphique.



Figure 24 : interface d'authentification

Toutefois si les informations entrées par l'utilisateur sont incorrect, le système lui renvoie la page d'authentification avec un message d'erreur.

2. Interfaces de tableaux de bord

Dans l'onglet "tableau de bords" nous avons un affichage du programme planifié, Le Chirurgienne à la possibilité de visualiser son planning et d'imprimer l'ensemble des programmes.

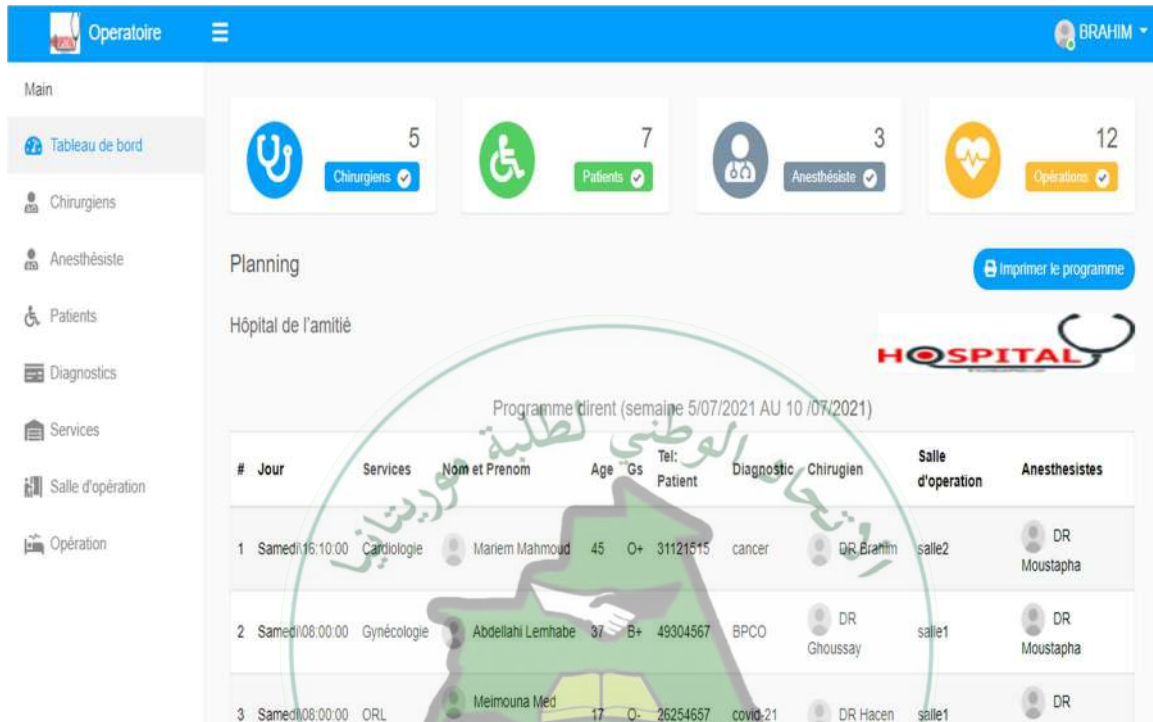


Figure 25 : Interfaces de tableau de bord

2.3. Interfaces de "chirurgiennes "

Ces interfaces offre à l'utilisateur l'accès à ses chirurgiennes et la possibilité de les gérer (ajouter, modifier, supprimer).

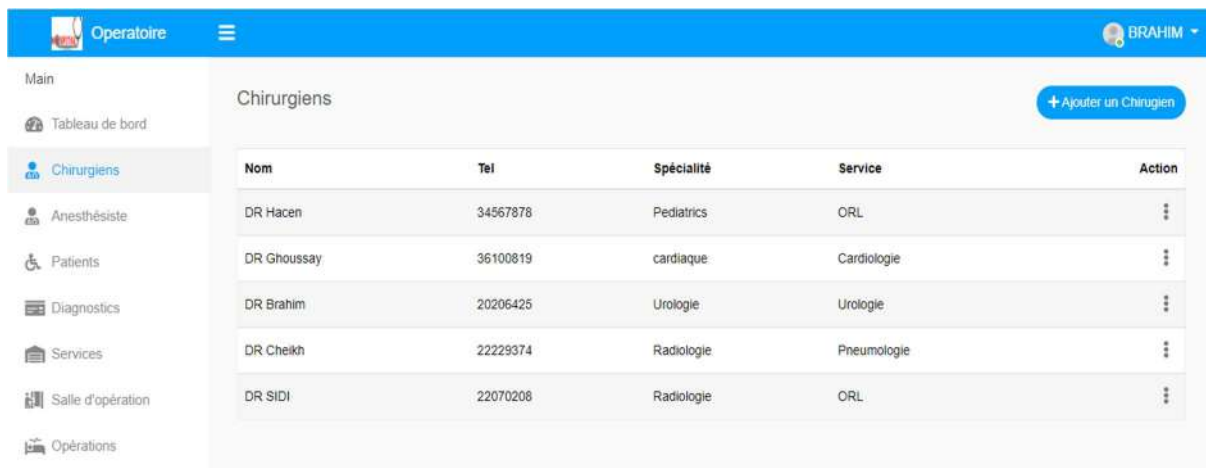


Figure 26 : Interface Chirurgiens

4. Interfaces de « Gestion des Anesthésistes »

Cet interface permet à l'utilisateur l'accès à l'Anesthésiste et la possibilité (ajouter, modifier, supprimer).

Figure 27: Interface de gestion des Anesthésistes

2.5. Interfaces de Gestion des Patients

Cette interface permet à l'utilisateur de gérer les patients et ayant la possibilité d'ajouter chaque patient dans un le programmes correspondant en lui attribuant (un diagnostic, un chirurgien, et une date d'opération).

#	Nom	Age	Groupe sang	Sexe	Tel	Diagnostic	Action
1	Isselmhoum Meilloud	46	O+	Female	26667878	covid-21	...
2	Abdallahi Naji	10	O+	Male	34466726	cancer	...
3	sidi mohamed	43	AB+	Male	34573543	covid-21	...
4	Mariam Mahmoud	45	O+	Female	31121515	cancer	...
5	Abdellahi Lemhabe	37	B+	Male	49304567	BPCO	...
6	Meimouna Med Salem	17	O-	Female	26254657	covid-21	...

Figure 28 : Interface patient

2.6. Interface de gestion des services

Cette interface permet à l'utilisateur de gérer les services.

#	Nom Service	Action
1	Neurochirurgie	...
2	Chirurgie viscérale	...
3	Gynécologie	...
4	Chirurgie plastique	...
5	Urologie	...
6	Pneumologie	...
7	Neurologie	...
8	Ophthalmologie	...

Figure 29 : interface de gestion des services

2.7. Interfaces Gestions des opérations

Ces interfaces permet à l'utilisateur l'accès à l'opération et la possibilité (ajouter, modifier, supprimer) une opération déjà planifier.

#	Anesthésiste	Salle d'opération	Jour d'opération	Heure	Action
1	DR Moustapha	salle1	2021-07-06	09:00:00	...
2	DR Moustapha	salle1	2021-07-07	16:10:00	...
3	DR Moustapha	salle1	2021-07-08	19:30:00	...
4	DR Moustapha	salle1	2021-08-23	08:45:00	...
5	DR Moustapha	salle1	2021-08-28	08:30:00	...
6	DR Moustapha	salle2	2021-08-20	15:30:00	...

Figure 30 : Interface des Operations

2. Conclusion :

En somme, ce chapitre nous a permis de rendre visible les aspects qui ont été évoqués dans le chapitre précédent. Il correspond à la dernière partie de ce rapport et a pour objet de clarifier toutes les fonctionnalités de l'application.

4. Conclusion Générale :

Au cours de ce travail, j'ai présenté les différentes étapes ayant conduit à la mise en œuvre ce système de gestion des programmes opératoires.

J'ai commencé par recenser les difficultés que rencontre les chirurgiens et les patient dans les hôpitaux pour une opération afin d'apporter une solution adéquate et spécifier ainsi les besoins.

Le langage de modélisation **UML** et le processus unifié **UP** ont constitués le support de l'analyse des besoins et la conception de mon application web via les différents diagrammes **UML** couvrant les aspects fonctionnels, dynamiques et statiques de tout le développement. Pour enfin réaliser l'application, j'ai utilisé le **SGBD MYSQL** pour implémenter la base de données et les Framework **laravel(PHP)**, **Bootstrab** ainsi les langages : **HTML**, **CSS**.

Pour conclure, ce projet a fait l'objet d'une expérience intéressante, très bénéfique pour moi. En effet, il m'a permis d'enrichir mes connaissances théoriques et compétences dans le domaine de la conception et de la programmation. Ajoutant à ceci, la mise en application des connaissances acquises tout au long de mon étude. Par ailleurs, j'ai noté qu'un projet de développement logiciel n'est jamais Complètement achevé, c'est à cet effet que nous envisageons au futur d'apporter des améliorations supplémentaires sur l'application. Sur ce nous comptons :

- Mettre en place un forum de discussion entre les Chirurgiens et les patients
- Mettre en place un processus opératoire qui sera décomposé en deux phases :
 - **Phase préopératoire** : Mettre en place un forum de discussion entre les médecins et les patients
 - **Phase post-opératoire** : Mettre en place un forum de discussion entre les médecins et les patients
- Mettre en places une estimation de la durée opératoire

WEBOGRAPHIE

<https://laravel.com/>

<https://code.visualstudio.com/docs>

https://developer.mozilla.org/fr/docs/Learn/JavaScript/First_steps/What_is_JavaScript

https://fr.wikipedia.org/wiki/Mod%C3%A8le_vuecontr%C3%B4leur

